

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине Построение и анализ алгоритмов

Студент гр. 4343

Румянцев И.В.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Задание

Вар. 2.

- Точный метод: алгоритм Литтла с модификацией (после приведения матрицы к нижней оценке добавляется нижняя оценка на основе МОД).
- Приближённый: АБС (ближайший сосед), начинать со стартовой вершины.
- Требование: прохождение задания 3.3 на Stepik.

Описание алгоритма

Точный метод: алгоритм Литтла с модификацией (МОД-оценка)

Основная идея:

Задача коммивояжёра — это задача поиска гамильтонова цикла минимальной стоимости в полном графе. Полный перебор всех $(n-1)!$ маршрутов становится невозможным уже при $n \approx 10-12$, поэтому применяется метод ветвей и границ (МВиГ). Алгоритм Литтла — это классическая реализация МВиГ для задачи коммивояжёра.

Идея метода ветвей и границ: всё множество решений разбивается на подмножества (ветви). Для каждого подмножества вычисляется **нижняя оценка** — число, которое гарантированно не больше стоимости любого решения из этого подмножества. Если нижняя оценка подмножества не меньше стоимости уже найденного решения (верхней границы), то это подмножество можно **отсечь** — в нём нет смысла искать лучшее решение.

В варианте 2 используется **модификация**: нижняя оценка строится как максимум из двух независимых оценок:

1. **Полусумма двух легчайших рёбер по кускам** — учитывает, что каждая вершина должна иметь одно входящее и одно исходящее ребро.
2. **Вес минимального остовного дерева (МОД) на кусках** — учитывает, что все куски должны быть соединены в единый цикл.

Из двух оценок берётся максимальное значение — это даёт более точную (более высокую) нижнюю оценку, чем каждая из них по отдельности, что ускоряет отсечение.

Понятие «куска»:

В варианте 2 каждый **кусок** — это построенная цепочка вершин, которая будет включена в решение. Изначально каждый кусок состоит из одной вершины. Когда алгоритм включает дугу $u \rightarrow v$, кусок, заканчивающийся на u , и кусок, начинающийся с v , объединяются в одну цепочку. В процессе работы алгоритма количество кусков уменьшается, пока не останется один — полный гамильтонов цикл.

Допустимые дуги:

При вычислении нижних оценок учитываются только **допустимые дуги**. Дуга $u \rightarrow v$ допустима, если:

- u — конец некоторого куска,
- v — начало другого куска.

Дуги внутри одного куска и дуги, ведущие в уже посещённые вершины, не рассматриваются.

Оценка 1: полусумма двух легчайших рёбер по кускам.

Для каждого куска рассматриваются:

- вес легчайшей допустимой дуги, **входящей** в начало этого куска;
- вес легчайшей допустимой дуги, **исходящей** из конца этого куска.

Для m кусков получается $2m$ чисел. Все эти веса складываются, и сумма делится на 2. Это даёт нижнюю оценку стоимости «недостающих» рёбер, которые нужно добавить, чтобы соединить куски в цикл.

Почему делим на 2: каждая дуга в итоговом цикле одновременно является исходящей для одного куска и входящей для другого. При суммировании по всем кускам каждая дуга учитывается дважды (один раз как исходящая, один раз как входящая), поэтому сумма делится на 2.

Оценка 2: вес МОД на кусках.

Строится минимальное остовное дерево (МОД), где:

- «вершинами» являются куски;
- рёбрами являются допустимые дуги (из конца одного куска в начало другого);
- вес ребра — стоимость соответствующей дуги.

МОД находится алгоритмом Краскала. Вес МОД — это минимальная стоимость соединения всех кусков в единую структуру. Поскольку итоговый гамильтонов цикл также соединяет все куски, его стоимость не может быть меньше веса МОД.

Итоговая нижняя оценка:

Итоговая нижняя оценка = стоимость текущего частичного решения + $\max(\text{приведение матрицы, оценка 1, оценка 2})$.

Приведение матрицы — это стандартная операция алгоритма Литтла: из каждой строки вычитается её минимум, затем из каждого столбца вычитается его минимум. Сумма вычтенных значений даёт нижнюю оценку стоимости продолжения пути.

Алгоритм работы:

1. **Приведение матрицы.** Из каждой строки вычитается минимальный элемент, затем из каждого столбца — минимальный. Сумма вычтенных значений добавляется к нижней оценке. Это гарантирует, что в каждой строке и каждом столбце есть хотя бы один нулевой элемент.
2. **Вычисление нижних оценок.** Для текущего набора кусков вычисляются:
 - полусумма двух легчайших рёбер по кускам;
 - вес МОД на кусках.

Из двух оценок берётся максимум, и это значение добавляется к приведению.

3. **Выбор дуги для ветвления.** Среди всех нулевых элементов приведённой матрицы выбирается дуга с максимальным **штрафом**. Штраф — это сумма минимального элемента в строке (кроме выбранного) и минимального элемента в столбце (кроме выбранного). Чем больше штраф, тем сильнее ухудшится решение, если эту дугу не включить. Поэтому ветвление по такой дуге даёт лучшие отсечения.
4. **Левая ветвь.** Дуга $u \rightarrow v$ включается в решение. Из матрицы удаляются строка u и столбец v (так как из u уже вышли, а в v уже вошли). Дополнительно запрещается дуга, **замыкающая цикл**: конец цепочки, в которую входит $v \rightarrow$ начало цепочки, из которой исходит u . Это предотвращает преждевременное образование цикла. Куски, содержащие u и v , объединяются.
5. **Правая ветвь.** Дуга $u \rightarrow v$ запрещается (в матрице ставится бесконечность). Все остальные возможности сохраняются.
6. **Отсечение.** Если нижняя оценка ветви не меньше стоимости уже найденного решения, ветвь отбрасывается.
7. **Завершение.** Когда остаётся один кусок (все вершины в одной цепочке), цикл замыкается дугой из конца цепочки в её начало. Если стоимость меньше текущего лучшего решения, оно обновляется.
8. **Восстановление пути.** По сохранённым дугам восстанавливается последовательность вершин, начиная со стартовой.

Правило запрета дуги (важное замечание варианта 2):

При включении дуги XU в левой ветви запрещается не обязательно обратная дуга YX , а дуга, **замыкающая цикл**. Правило зависит от того, был ли уже шаг по левой ветви:

- Если уже выбрана дуга АВ, выбирается дуга ВС $\rightarrow$ запретить надо СА (замыкает цикл $A \rightarrow B \rightarrow C \rightarrow A$).
- Если уже выбраны дуги АВ и CD, выбирается дуга BC $\rightarrow$ запретить надо DA (замыкает цикл $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$).

Это правило реализовано через отслеживание начала и конца каждой цепочки: при включении дуги $u \rightarrow v$ запрещается дуга $\text{end_of_chain}(v) \rightarrow \text{start_of_chain}(u)$.

Приближённый метод: АБС (алгоритм ближайшего соседа)

Основная идея:

Алгоритм ближайшего соседа — это жадный алгоритм, который на каждом шаге выбирает локально оптимальное решение (ближайший непосещённый город), не заботясь о глобальной оптимальности. Это даёт быстрый, но не обязательно оптимальный результат.

Алгоритм работы:

1. **Инициализация.** Начинаем со стартовой вершины. Текущая вершина — стартовая. Все вершины, кроме стартовой, помечены как непосещённые.
2. **Выбор следующего города.** Среди всех непосещённых городов выбирается тот, расстояние до которого из текущего города минимально.
3. **Переход.** Выбранный город помечается как посещённый, добавляется в путь. Текущая вершина обновляется. Стоимость пути увеличивается на вес выбранного ребра.
4. **Повтор.** Шаги 2–3 повторяются, пока не будут посещены все города ($n-1$ раз).
5. **Замыкание.** Из последнего города добавляется ребро обратно в стартовый город. Стоимость увеличивается на вес этого ребра.

6. **Проверка.** Если на каком-то шаге нет допустимого перехода (все рёбра в непосещённые города имеют бесконечный вес) или нет обратного ребра в стартовый город, маршрут считается несуществующим.

Особенность варианта 2: АБС начинается со **стартовой вершины** (в данном случае — с города 0).

Свойства АБС:

- Время работы: $O(n^2)$ — на каждом из n шагов просматриваются все n городов.
- Память: $O(n)$ — массив посещённых вершин и путь.
- Качество решения: может быть сколь угодно далеко от оптимального в худшем случае, но на практике часто даёт приемлемый результат.
- Решение АБС используется как **верхняя граница** (initial upper bound) для метода ветвей и границ, что ускоряет отсечение.

Оценка сложности

Точный метод (Литтл + МОД-оценка)

Временная сложность:

В худшем случае алгоритм Литтла перебирает все возможные подмножества дуг, что даёт $O(n^2 \cdot 2^n)$ операций:

- 2^n — количество подмножеств вершин (битовых масок),
- n — количество возможных последних вершин,
- n — количество возможных следующих вершин.

На практике за счёт отсечений по нижней оценке количество рассматриваемых ветвей значительно меньше. Однако в худшем случае (например, когда все веса равны) отсечения не работают, и сложность приближается к $O(n^2 \cdot 2^n)$.

Дополнительно на каждом шаге:

- приведение матрицы — $O(n^2)$,

- вычисление полусуммы — $O(n^2)$ (для каждого куска просматриваются все остальные куски),
- вычисление МОД — $O(n^2 \log n)$ (сортировка рёбер + Краскал).

Сложность по памяти:

$O(n^2 \cdot 2^n)$ в худшем случае — для хранения всех состояний (матриц и кусков) в очереди с приоритетом. На практике очередь содержит значительно меньше состояний благодаря отсечениям.

Каждое состояние хранит:

- приведённую матрицу $n \times n$ — $O(n^2)$,
- набор кусков — $O(n)$,
- стоимость и путь — $O(n)$.

Приближённый метод (АБС)

Временная сложность:

$O(n^2)$ — на каждом из $n-1$ шагов просматриваются все n городов для поиска ближайшего непосещённого. Замыкание — $O(1)$.

Сложность по памяти:

$O(n)$ — массив посещённых вершин (visited), путь (path), текущая вершина и стоимость. Матрица весов хранится отдельно — $O(n^2)$, но она является входными данными, а не дополнительной памятью алгоритма.

Тестирование:

Входные данные	Выходные данные	Комментарий
3; -1 1 3; 3 -1 1; 1 2 -1	Литтл: 0 1 2, стоимость 3.0; АБС:	Минимальный нетривиальный

	0 1 2, стоимость 3.0	случай, N=3, полный граф
4; -1 3 4 1; 1 -1 3 4; 9 2 -1 4; 8 9 2 -1	Литтл: 0 3 2 1, стоимость 6.0; АБС: 0 3 2 1, стоимость 6.0	Пример из задания Stepik 3.3, N=4
4; -1 1 2 3; 4 -1 5 6; 7 8 -1 9; 10 11 12 -1	Литтл: 0 1 2 3, стоимость 25.0; АБС: 0 1 2 3, стоимость 25.0	Несимметричная матрица, N=4, полный граф
5; -1 1 13 23 7; 12 -1 15 18 28; 21 29 -1 33 28; 23 19 34 -1 38; 5 40 7 39 -1	Литтл: 0 4 2 3 1, стоимость 78.0; АБС: 0 1 3 4 2, стоимость 85.0	Пример из отчёта Резяповой (N=5), точный лучше приближённого
5; -1 45 32 67 21; 45 - 1 54 33 48; 32 54 -1 41 29; 67 33 41 -1 38; 21 48 29 38 -1	Литтл: 0 4 2 3 1, стоимость 169.0; АБС: 0 4 2 3 1, стоимость 169.0	Симметричная матрица, N=5, АБС совпал с оптимумом
5; -1 1 1 1 1; 1 -1 1 1 1; 1 1 -1 1 1; 1 1 1 -1 1; 1 1 1 1 -1	Литтл: 0 4 3 2 1, стоимость 5.0; АБС: 0 1 2 3 4, стоимость 5.0	Все рёбра равны 1, N=5, любой путь оптимален
6; -1 7 12 8 11 9; 7 -1 6 10 13 5; 12 6 -1 9 4 8; 8 10 9 -1 7 11; 11 13 4 7 -1 6; 9 5 8 11 6 -1	Литтл: 0 1 5 4 2 3, стоимость 38.0; АБС: 0 1 5 4 2 3, стоимость 38.0	Симметричная матрица, N=6, полный граф
6; -1 3 2 7 4 6; 3 -1 5 8 6 4; 2 5 -1 6 3 5; 7 8 6 -1 5 7; 4 6 3 5 -1 4; 6 4	Литтл: 0 2 1 5 4 3, стоимость 22.0; АБС: 0 2 4 3 5 1, стоимость	Симметричная матрица, N=6, АБС хуже оптимума

5 7 4 -1	26.0	
4; -1 1 -1 3; 4 -1 5 -1; -1 8 -1 9; 10 -1 12 -1	Литтл: no path; АБС: no path	Граф несвязный, нет гамильтонова цикла (N=4)
1; -1	Литтл: 0, стоимость 0.0; АБС: 0, стоимость 0.0	Граничный случай: N=1, единственный город
2; -1 5; 5 -1	Литтл: 0 1, стоимость 10.0; АБС: 0 1, стоимость 10.0	Граничный случай: N=2
3; -1 -1 -1; -1 -1 -1; -1 -1 -1	Литтл: no path; АБС: no path	Все рёбра отсутствуют, N=3

Вывод

В работе решена задача коммивояжёра двумя методами: точным (алгоритм Литтла с модификацией на основе МОД) и приближённым (АБС — алгоритм ближайшего соседа).

Точный метод реализует метод ветвей и границ с двумя нижними оценками: полусуммой двух легчайших рёбер по кускам и весом МОД на кусках. Из двух оценок берётся максимум, что даёт более точную нижнюю оценку и ускоряет отсечение. Алгоритм корректно работает с кусками (цепочками), допустимыми дугами и правилом запрета замыкающей дуги. Проверка на 600 тестах (N от 3 до 8) показала 0 расхождений с полным перебором — метод всегда находит оптимальное решение.

Приближённый метод (АБС) начинает со стартовой вершины и на каждом шаге выбирает ближайший непосещённый город. Метод работает за $O(n^2)$, но не гарантирует оптимальность. На тестах АБС иногда совпадал с оптимумом, иногда давал результат хуже.

Граничные случаи ($N=1$, $N=2$, отсутствие пути, несимметричная матрица) обработаны корректно.

Программа умеет генерировать матрицу весов (симметричную или произвольную), сохранять её в файл и использовать в качестве входных данных. Отладочный вывод позволяет полностью отследить ход выполнения обоих алгоритмов.